

Java Quick Refresher

3-4 line problem -> identify -> solution prompts

Use this as a fast review sheet before interviews or revision sessions.

AnonymousClass

Anonymous classes, interfaces, and small behavior overrides.

Problem	Identify -> solve	Mental model
AnonymousClass	Anonymous Class: anonymous classes are local behavior containers.	The lesson is to keep small behaviour close to where it is used.
Emp	Emp: anonymous classes are local behavior containers.	The lesson is to keep small behaviour close to where it is used.
Int	Int: anonymous classes are local behavior containers.	The lesson is to keep small behaviour close to where it is used.
Inter	Inter: anonymous classes are local behavior containers.	The lesson is to keep small behaviour close to where it is used.
Test	Test: anonymous classes are local behavior containers.	The lesson is to keep small behaviour close to where it is used.
Test1	Test1: anonymous classes are local behavior containers.	The lesson is to keep small behaviour close to where it is used.
Test2	Test2: anonymous classes are local behavior containers.	The lesson is to keep small behaviour close to where it is used.

Number_Theory

Digit math and elementary number-theory style manipulation.

Problem	Identify -> solve	Mental model
Add_Digits	Add Digits: number theory problems often collapse to digit invariants.	The lesson is to recognize digit invariants and avoid full expansion.

Queue

Queue fundamentals and linear FIFO data flow.

Problem	Identify -> solve	Mental model
BasicsOfQueue	Basics Of Queue: queues are about ordering, not sorting.	The lesson is that queues model first-in-first-out workflows.
QueueUsingArray	Queue Using Array: queues are about ordering, not sorting.	The lesson is that queues model first-in-first-out workflows.

Serialization

Object persistence and custom serialization concerns.

Problem	Identify -> solve	Mental model
Bean	Bean: serialization is about durable object state.	The lesson is to treat object persistence as a contract, not a side effect.
Serialize	Serialize: serialization is about durable object state.	The lesson is to treat object persistence as a contract, not a side effect.

Thread

Concurrency, coordination, synchronization, and lock-based sequencing.

Problem	Identify -> solve	Mental model
Lock	Lock: thread problems teach safe coordination, not raw speed.	The lesson is that concurrency problems are about state ownership and ordering guarantees.
OddEvenPrinter	Odd Even Printer: thread problems teach safe coordination, not raw speed.	The lesson is that concurrency problems are about state ownership and ordering guarantees.
ReentrantLockExample	Reentrant Lock Example: thread problems teach safe coordination, not raw speed.	The lesson is that concurrency problems are about state ownership and ordering guarantees.
Thread1	Thread1: thread problems teach safe coordination, not raw speed.	The lesson is that concurrency problems are about state ownership and ordering guarantees.
Thread2	Thread2: thread problems teach safe coordination, not raw speed.	The lesson is that concurrency problems are about state ownership and ordering guarantees.
ThreadMain	Thread Main: thread problems teach safe coordination, not raw speed.	The lesson is that concurrency problems are about state ownership and ordering guarantees.

arrays

Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.

Problem	Identify -> solve	Mental model
Anagram	Anagram: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Anagram is to train the eye to spot the topic-level pattern quickly.
ArrayDivision	Array Division: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Array Division is to train the eye to spot the topic-level pattern quickly.
BooksAllocation	Books Allocation: greedy works when a safe local decision stays optimal.	The lesson is to ask whether the problem is about proving a safe local choice.
CanPlaceFlower	Can Place Flower: greedy works when a safe local decision stays optimal.	The lesson is to ask whether the problem is about proving a safe local choice.
ChocolateDistribution	Chocolate Distribution: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Chocolate Distribution is to train the eye to spot the topic-level pattern quickly.
ChocolatePickup_Hard	Chocolate Pickup Hard: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Chocolate Pickup Hard is to train the eye to spot the topic-level pattern quickly.
Classmates_StoreTwoNumbers	Classmates Store Two Numbers: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Classmates Store Two Numbers is to train the eye to spot the topic-level pattern quickly.
ClosestPairFromSortedArray	Closest Pair From Sorted Array: the key question is which sort matches the constraints.	The lesson is to choose a sort based on data structure and constraints, not habit.
CombinationSum_24_08	Combination Sum 24 08: contiguous range problems usually reward prefix or window state.	The lesson is to recognize when a range query can be turned into incremental state.
CombinationSum_25_08	Combination Sum 25 08: contiguous range problems usually reward prefix or window state.	The lesson is to recognize when a range query can be turned into incremental state.
Complement_kadane	Complement kadane: the trick is usually an invariant that survives a single sweep.	The lesson is to reduce the problem to an invariant that can survive one pass.
CountTriplets	Count Triplets: contiguous range problems usually reward prefix or window state.	The lesson is to recognize when a range query can be turned into incremental state.
DistinctElementsInEveryWindow	Distinct Elements In Every Window: contiguous range problems usually reward prefix or window state.	The lesson is to recognize when a range query can be turned into incremental state.
DutchNationalFlag	Dutch National Flag: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Dutch National Flag is to train the eye to spot the topic-level pattern quickly.
EnhancedMergeSort	Enhanced Merge Sort: the key question is which sort matches the constraints.	The lesson is to choose a sort based on data structure and constraints, not habit.
EquilibriumPoint	Equilibrium Point: contiguous range problems usually reward prefix or window state.	The lesson is to recognize when a range query can be turned into incremental state.
FindMinimumInRotatedSortedArray	Find Minimum In Rotated Sorted Array: the key question is which sort matches the constraints.	The lesson is to choose a sort based on data structure and constraints, not habit.

FindAllPermutations_03_09	Find All Permutations 03 09: recursive search with backtracking is the natural fit.	The lesson is to recognize decision trees and control the explosion with constraints.
FindMinInSortedArray	Find Min In Sorted Array: the key question is which sort matches the constraints.	The lesson is to choose a sort based on data structure and constraints, not habit.
FindSubarrayWithGivenSum	Find Subarray With Given Sum: contiguous range problems usually reward prefix or window state.	The lesson is to recognize when a range query can be turned into incremental state.
FourSum	Four Sum: contiguous range problems usually reward prefix or window state.	The lesson is to recognize when a range query can be turned into incremental state.
GoldMine	Gold Mine: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Gold Mine is to train the eye to spot the topic-level pattern quickly.
HouseRobber	House Robber: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of House Robber is to train the eye to spot the topic-level pattern quickly.
JobScheduling	Job Scheduling: greedy works when a safe local decision stays optimal.	The lesson is to ask whether the problem is about proving a safe local choice.
JosephusProblem	Josephus Problem: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Josephus Problem is to train the eye to spot the topic-level pattern quickly.
KMP_pattern	K M P pattern: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of K M P pattern is to train the eye to spot the topic-level pattern quickly.
LargestNumberFormedFromArray	Largest Number Formed From Array: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Largest Number Formed From Array is to train the eye to spot the topic-level pattern quickly.
LeadersInArray	Leaders In Array: the trick is usually an invariant that survives a single sweep.	The lesson is to reduce the problem to an invariant that can survive one pass.
LeftElementSmall	Left Element Small: contiguous range problems usually reward prefix or window state.	The lesson is to recognize when a range query can be turned into incremental state.
LongestCommonSubsequence	Longest Common Subsequence: contiguous range problems usually reward prefix or window state.	The lesson is to recognize when a range query can be turned into incremental state.
LongestConsecutiveSubsequence	Longest Consecutive Subsequence: contiguous range problems usually reward prefix or window state.	The lesson is to recognize when a range query can be turned into incremental state.
LongestIncreasingSubsequence	Longest Increasing Subsequence: contiguous range problems usually reward prefix or window state.	The lesson is to recognize when a range query can be turned into incremental state.
LongestSubArrayOfSumK	Longest Sub Array Of Sum K: contiguous range problems usually reward prefix or window state.	The lesson is to recognize when a range query can be turned into incremental state.
MajorityElement2	Majority Element2: the trick is usually an invariant that survives a single sweep.	The lesson is to reduce the problem to an invariant that can survive one pass.
MajorityElement_BoyerMoore	Majority Element Boyer Moore: the trick is usually an invariant that survives a single sweep.	The lesson is to reduce the problem to an invariant that can survive one pass.
MatrixChainMultiplication	Matrix Chain Multiplication: matrix/sequence problems often hinge	The lesson is to respect the required traversal order instead of reshaping data

	on traversal order.	unnecessarily.
MaxAvgSubarray	Max Avg Subarray: monotonicity is the signal that binary search applies.	The lesson is to ask whether the problem has a monotonic predicate.
MaxSumSubarray_kadane	Max Sum Subarray kadane: contiguous range problems usually reward prefix or window state.	The lesson is to recognize when a range query can be turned into incremental state.
MaximumSumOfNonAdjacentArray	Maximum Sum Of Non Adjacent Array: contiguous range problems usually reward prefix or window state.	The lesson is to recognize when a range query can be turned into incremental state.
MergeSortedArrays	Merge Sorted Arrays: the key question is which sort matches the constraints.	The lesson is to choose a sort based on data structure and constraints, not habit.
MinimumNumberOfTaps	Minimum Number Of Taps: monotonicity is the signal that binary search applies.	The lesson is to ask whether the problem has a monotonic predicate.
MinimumPlatforms	Minimum Platforms: monotonicity is the signal that binary search applies.	The lesson is to ask whether the problem has a monotonic predicate.
MissingElementsInARange	Missing Elements In A Range: the trick is usually an invariant that survives a single sweep.	The lesson is to reduce the problem to an invariant that can survive one pass.
MissingNumberInArray	Missing Number In Array: the trick is usually an invariant that survives a single sweep.	The lesson is to reduce the problem to an invariant that can survive one pass.
MissingNumbers	Missing Numbers: the trick is usually an invariant that survives a single sweep.	The lesson is to reduce the problem to an invariant that can survive one pass.
NQueens	N Queens: recursive search with backtracking is the natural fit.	The lesson is to recognize decision trees and control the explosion with constraints.
Ninja_Training	Ninja Training: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Ninja Training is to train the eye to spot the topic-level pattern quickly.
NumberOfWaysInMatrix	Number Of Ways In Matrix: matrix/sequence problems often hinge on traversal order.	The lesson is to respect the required traversal order instead of reshaping data unnecessarily.
NumberOfWaysToReachDestination	Number Of Ways To Reach Destination: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Number Of Ways To Reach Destination is to train the eye to spot the topic-level pattern quickly.
OverlappingIntervals	Overlapping Intervals: greedy works when a safe local decision stays optimal.	The lesson is to ask whether the problem is about proving a safe local choice.
PalindromePartitioning	Palindrome Partitioning: recursive search with backtracking is the natural fit.	The lesson is to recognize decision trees and control the explosion with constraints.
PalindromeString	Palindrome String: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Palindrome String is to train the eye to spot the topic-level pattern quickly.
PermutationsOfString	Permutations Of String: recursive search with backtracking is the natural fit.	The lesson is to recognize decision trees and control the explosion with constraints.
PrintSolutionsOfNQueen	Print Solutions Of N Queen: recursive search with backtracking is the natural fit.	The lesson is to recognize decision trees and control the explosion with constraints.
PrintSubarrayWithGivenSum_22_07_24	Print Subarray With Given Sum 22 07 24: contiguous range problems usually reward prefix or window state.	The lesson is to recognize when a range query can be turned into incremental state.
ProductOfArrayExceptSelf_24_10	Product Of Array Except Self 24 10: the trick is usually an invariant that survives a single sweep.	The lesson is to reduce the problem to an invariant that can survive one pass.

PythagoreanTriplet	Pythagorean Triplet: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Pythagorean Triplet is to train the eye to spot the topic-level pattern quickly.
RabinKarp	Rabin Karp: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Rabin Karp is to train the eye to spot the topic-level pattern quickly.
RainWaterTrapping	Rain Water Trapping: the trick is usually an invariant that survives a single sweep.	The lesson is to reduce the problem to an invariant that can survive one pass.
ReachNthStep	Reach Nth Step: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Reach Nth Step is to train the eye to spot the topic-level pattern quickly.
RearrangeArrayElements	Rearrange Array Elements: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Rearrange Array Elements is to train the eye to spot the topic-level pattern quickly.
ReverseANumber	Reverse A Number: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Reverse A Number is to train the eye to spot the topic-level pattern quickly.
ReverseArrayinGroups	Reverse Arrayin Groups: matrix/sequence problems often hinge on traversal order.	The lesson is to respect the required traversal order instead of reshaping data unnecessarily.
SearchInRotatedSortedArray	Search In Rotated Sorted Array: the key question is which sort matches the constraints.	The lesson is to choose a sort based on data structure and constraints, not habit.
Sieve_GCD	Sieve G C D: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Sieve G C D is to train the eye to spot the topic-level pattern quickly.
SlidingWindowMaximum	Sliding Window Maximum: contiguous range problems usually reward prefix or window state.	The lesson is to recognize when a range query can be turned into incremental state.
SortedInfiniteArray	Sorted Infinite Array: the key question is which sort matches the constraints.	The lesson is to choose a sort based on data structure and constraints, not habit.
SpirallyTraverseMatrix	Spirally Traverse Matrix: matrix/sequence problems often hinge on traversal order.	The lesson is to respect the required traversal order instead of reshaping data unnecessarily.
StockBuySell	Stock Buy Sell: greedy works when a safe local decision stays optimal.	The lesson is to ask whether the problem is about proving a safe local choice.
StockBuySell2	Stock Buy Sell2: greedy works when a safe local decision stays optimal.	The lesson is to ask whether the problem is about proving a safe local choice.
StockBuySellAtmostKTransaction	Stock Buy Sell Atmost K Transaction: greedy works when a safe local decision stays optimal.	The lesson is to ask whether the problem is about proving a safe local choice.
Store_2_Integers_At_One_Index	Store 2 Integers At One Index: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Store 2 Integers At One Index is to train the eye to spot the topic-level pattern quickly.
StringCompression	String Compression: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of String Compression is to train the eye to spot the topic-level pattern quickly.
SubArrayWithGivenSum	Sub Array With Given Sum: contiguous range problems usually reward prefix or window state.	The lesson is to recognize when a range query can be turned into incremental state.

SubArraysWithOddSum	Sub Arrays With Odd Sum: contiguous range problems usually reward prefix or window state.	The lesson is to recognize when a range query can be turned into incremental state.
SubSetSum_1__28_08_24	Sub Set Sum 1 28 08 24: contiguous range problems usually reward prefix or window state.	The lesson is to recognize when a range query can be turned into incremental state.
SubarrayOfSumK	Subarray Of Sum K: contiguous range problems usually reward prefix or window state.	The lesson is to recognize when a range query can be turned into incremental state.
SubsetOfString	Subset Of String: recursive search with backtracking is the natural fit.	The lesson is to recognize decision trees and control the explosion with constraints.
SumOfInfiniteArray	Sum Of Infinite Array: contiguous range problems usually reward prefix or window state.	The lesson is to recognize when a range query can be turned into incremental state.
UniquePaths_	Unique Paths: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Unique Paths is to train the eye to spot the topic-level pattern quickly.
ZigZag	Zig Zag: matrix/sequence problems often hinge on traversal order.	The lesson is to respect the required traversal order instead of reshaping data unnecessarily.
ZigzagConversion	Zigzag Conversion: matrix/sequence problems often hinge on traversal order.	The lesson is to respect the required traversal order instead of reshaping data unnecessarily.

binaryTree

Tree recursion, traversal patterns, path problems, BST reasoning, and tree transformation.

Problem	Identify -> solve	Mental model
BFS_DFS	B F S D F S: the main choice is DFS versus BFS, then carrying just enough state.	The lesson is to match traversal strategy to the output shape.
BinaryTree	Binary Tree: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Binary Tree is to train the eye to spot the topic-level pattern quickly.
BurnATree	Burn A Tree: the key is whether the shape should change locally or level-wise.	The lesson is to treat the tree as a structure to be rewired, not flattened into arrays unless required.
CheckIfSubtree	Check If Subtree: recursion answers where the targets are, so the split point becomes obvious.	The lesson is that tree ancestry can usually be solved by letting recursion report presence upward.
ConstructTreeFromViews	Construct Tree From Views: the key is whether the shape should change locally or level-wise.	The lesson is to treat the tree as a structure to be rewired, not flattened into arrays unless required.
ConvertSortedArrayToBst	Convert Sorted Array To Bst: the key is whether the shape should change locally or level-wise.	The lesson is to treat the tree as a structure to be rewired, not flattened into arrays unless required.
DeepestLeavesSum	Deepest Leaves Sum: return one number to the parent and keep the true answer separately.	The lesson is that tree DP often means 'return one value upward, keep another value globally.'
DiameterOfTree	Diameter Of Tree: return one number to the parent and keep the true answer separately.	The lesson is that tree DP often means 'return one value upward, keep another value globally.'
FlattenABinaryTree	Flatten A Binary Tree: the key is whether the shape should change locally or level-wise.	The lesson is to treat the tree as a structure to be rewired, not flattened into arrays unless required.
HeightOfTree	Height Of Tree: the key is whether the shape should change locally or level-wise.	The lesson is to treat the tree as a structure to be rewired, not flattened into arrays unless required.
InvertABinaryTree	Invert A Binary Tree: the key is whether the shape should change locally or level-wise.	The lesson is to treat the tree as a structure to be rewired, not flattened into arrays unless required.
IsCousins	Is Cousins: recursion answers where the targets are, so the split point becomes obvious.	The lesson is that tree ancestry can usually be solved by letting recursion report presence upward.
LargestPathBetweenLeafNodes	Largest Path Between Leaf Nodes: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Largest Path Between Leaf Nodes is to train the eye to spot the topic-level pattern quickly.
LevelOrderTraversal	Level Order Traversal: the main choice is DFS versus BFS, then carrying just enough state.	The lesson is to match traversal strategy to the output shape.
LowestCommonAncestor	Lowest Common Ancestor: recursion answers where the targets are, so the split point becomes obvious.	The lesson is that tree ancestry can usually be solved by letting recursion report presence upward.
MajorityElement_BoyerMoore	Majority Element Boyer Moore: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Majority Element Boyer Moore is to train the eye to spot the topic-level pattern quickly.
MaxPathSum	Max Path Sum: prefix state turns repeated path checks into one DFS pass.	The lesson is to decide whether you need a boolean, a path, or a count before choosing the recursion state.

MaxSumOfRootToLeafPaths	Max Sum Of Root To Leaf Paths: return one number to the parent and keep the true answer separately.	The lesson is that tree DP often means 'return one value upward, keep another value globally.'
MinDepth	Min Depth: the key is whether the shape should change locally or level-wise.	The lesson is to treat the tree as a structure to be rewired, not flattened into arrays unless required.
Min_Absolute_Diff	Min Absolute Diff: the key is whether the shape should change locally or level-wise.	The lesson is to treat the tree as a structure to be rewired, not flattened into arrays unless required.
Node	Node: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Node is to train the eye to spot the topic-level pattern quickly.
PathSum	Path Sum: prefix state turns repeated path checks into one DFS pass.	The lesson is to decide whether you need a boolean, a path, or a count before choosing the recursion state.
PathSum2	Path Sum2: prefix state turns repeated path checks into one DFS pass.	The lesson is to decide whether you need a boolean, a path, or a count before choosing the recursion state.
PathSum3	Path Sum3: prefix state turns repeated path checks into one DFS pass.	The lesson is to decide whether you need a boolean, a path, or a count before choosing the recursion state.
SizeOfATree_Min_Max	Size Of A Tree Min Max: the key is whether the shape should change locally or level-wise.	The lesson is to treat the tree as a structure to be rewired, not flattened into arrays unless required.
SortedArrayToBst	Sorted Array To Bst: the key is whether the shape should change locally or level-wise.	The lesson is to treat the tree as a structure to be rewired, not flattened into arrays unless required.
StandardTree	Standard Tree: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Standard Tree is to train the eye to spot the topic-level pattern quickly.
SwappedNodeOfBST	Swapped Node Of B S T: the key is whether the shape should change locally or level-wise.	The lesson is to treat the tree as a structure to be rewired, not flattened into arrays unless required.
TiltOfATree	Tilt Of A Tree: return one number to the parent and keep the true answer separately.	The lesson is that tree DP often means 'return one value upward, keep another value globally.'
Traversal	Traversal: the main choice is DFS versus BFS, then carrying just enough state.	The lesson is to match traversal strategy to the output shape.
Tree	Tree: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Tree is to train the eye to spot the topic-level pattern quickly.
Vaccine	Vaccine: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Vaccine is to train the eye to spot the topic-level pattern quickly.
View	View: the key is whether the shape should change locally or level-wise.	The lesson is to treat the tree as a structure to be rewired, not flattened into arrays unless required.
ViewOfTree	View Of Tree: the key is whether the shape should change locally or level-wise.	The lesson is to treat the tree as a structure to be rewired, not flattened into arrays unless required.
ZigZagTraversalOfTree	Zig Zag Traversal Of Tree: the main choice is DFS versus BFS, then carrying just enough state.	The lesson is to match traversal strategy to the output shape.
isSameTree	is Same Tree: recursion answers where the targets are, so the split point	The lesson is that tree ancestry can usually be solved by letting recursion

	becomes obvious.	report presence upward.
--	------------------	-------------------------

builder

Builder pattern and object construction with readability and immutability in mind.

Problem	Identify -> solve	Mental model
CreatePizzaWithBuilder	Create Pizza With Builder: builder reduces constructor noise and improves clarity.	The lesson is to separate object creation from object use.
CreatePizzaWithoutBuilder	Create Pizza Without Builder: builder reduces constructor noise and improves clarity.	The lesson is to separate object creation from object use.
PizzaWithBuilder	Pizza With Builder: builder reduces constructor noise and improves clarity.	The lesson is to separate object creation from object use.
PizzaWithoutBuilder	Pizza Without Builder: builder reduces constructor noise and improves clarity.	The lesson is to separate object creation from object use.

consumer_supplier

Functional interface usage and callback-style APIs.

Problem	Identify -> solve	Mental model
ConsumerTest	Consumer Test: the abstraction is the direction of data flow.	The lesson is to separate data generation from data use.

dynamic

Dynamic programming fundamentals and optimization by storing subproblem results.

Problem	Identify -> solve	Mental model
KnapSack	Knap Sack: DP starts with the state, not with loops.	The lesson is to define the state before you define the code.
MinCoins	Min Coins: DP starts with the state, not with loops.	The lesson is to define the state before you define the code.
UglyNumbers	Ugly Numbers: DP starts with the state, not with loops.	The lesson is to define the state before you define the code.

graph

Traversal, shortest path, minimum spanning tree, cycle detection, and grid-to-graph patterns.

Problem	Identify -> solve	Mental model
BFS	B F S: traversal with a visited set prevents repeated exploration.	The lesson is to detect reachability and connected components with one systematic traversal.
BellmanFord	Bellman Ford: the edge constraints decide the algorithm, not the node count alone.	The lesson is to match the shortest-path method to the weight rules.
CycleDetection	Cycle Detection: cycles are found by distinguishing 'seen before' from 'seen on current path'.	The lesson is that cycle detection is about path state, not just global visited status.
DFS	D F S: traversal with a visited set prevents repeated exploration.	The lesson is to detect reachability and connected components with one systematic traversal.
DijkstraAlgorithm	Dijkstra Algorithm: the edge constraints decide the algorithm, not the node count alone.	The lesson is to match the shortest-path method to the weight rules.
FloydWarshall	Floyd Warshall: the edge constraints decide the algorithm, not the node count alone.	The lesson is to match the shortest-path method to the weight rules.
Graph	Graph: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Graph is to train the eye to spot the topic-level pattern quickly.
KruskalMST	Kruskal M S T: greedy edge selection is enough when the cut property holds.	The lesson is that MST problems are greedy because each safe local choice preserves global optimality.
NumberOfIslands	Number Of Islands: traversal with a visited set prevents repeated exploration.	The lesson is to detect reachability and connected components with one systematic traversal.
PathExistence	Path Existence: traversal with a visited set prevents repeated exploration.	The lesson is to detect reachability and connected components with one systematic traversal.
PrimMST	Prim M S T: greedy edge selection is enough when the cut property holds.	The lesson is that MST problems are greedy because each safe local choice preserves global optimality.
SnakeLadderProblem	Snake Ladder Problem: traversal with a visited set prevents repeated exploration.	The lesson is to detect reachability and connected components with one systematic traversal.

heap

Heap mechanics, priority ordering, top-k queries, and heap transformation patterns.

Problem	Identify -> solve	Mental model
DriveCar	DriveCar: one scan is enough; avoid heap or sort when the goal is just a max excess.	The lesson is to prefer direct aggregation when the output is a single statistic.
Heapify	Heapify: fix children first, then parent; that is why build-heap is linear.	The lesson is that local fixes composed bottom-up can produce a globally correct heap efficiently.
InsertAndDelete	Insert/Delete: only the affected path changes, so update in logarithmic time.	The lesson is that heaps are dynamic priority structures, not sorted arrays.
MinToMaxHeap	MinToMaxHeap: global validity comes from fixing all subtrees, not just the root.	The lesson is that heap validity is global; one visible fix is not enough.

leetCode

Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.

Problem	Identify -> solve	Mental model
AddCharToMakeSubsequence	Add Char To Make Subsequence: contiguous range problems usually reward prefix or window state.	The lesson is to recognize when a range query can be turned into incremental state.
AddTwoNumbers	Add Two Numbers: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Add Two Numbers is to train the eye to spot the topic-level pattern quickly.
BinarySearch	Binary Search: monotonicity is the signal that binary search applies.	The lesson is to ask whether the problem has a monotonic predicate.
Btree	Btree: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Btree is to train the eye to spot the topic-level pattern quickly.
ConstructProductMatrix	Construct Product Matrix: the trick is usually an invariant that survives a single sweep.	The lesson is to reduce the problem to an invariant that can survive one pass.
ContainerWithMostWater	Container With Most Water: the trick is usually an invariant that survives a single sweep.	The lesson is to reduce the problem to an invariant that can survive one pass.
CountServersThatCommunicate	Count Servers That Communicate: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Count Servers That Communicate is to train the eye to spot the topic-level pattern quickly.
DetonateMaximumBomb	Detonate Maximum Bomb: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Detonate Maximum Bomb is to train the eye to spot the topic-level pattern quickly.
FindFirstOccurence	Find First Occurrence: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Find First Occurrence is to train the eye to spot the topic-level pattern quickly.
FindAllAnagrams	Find All Anagrams: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Find All Anagrams is to train the eye to spot the topic-level pattern quickly.
FrogJump	Frog Jump: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Frog Jump is to train the eye to spot the topic-level pattern quickly.
FrogJumpWithKSteps	Frog Jump With K Steps: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Frog Jump With K Steps is to train the eye to spot the topic-level pattern quickly.
GameOfLife	Game Of Life: matrix/sequence problems often hinge on traversal order.	The lesson is to respect the required traversal order instead of reshaping data unnecessarily.
GasStation	Gas Station: greedy works when a safe local decision stays optimal.	The lesson is to ask whether the problem is about proving a safe local choice.
GenerateParentheses	Generate Parentheses: recursive search with backtracking is the natural fit.	The lesson is to recognize decision trees and control the explosion with constraints.
GetRandom	Get Random: identify the repeated work, choose the topic structure, and reduce	The goal of Get Random is to train the eye to spot the topic-level pattern

	the state until the answer becomes direct.	quickly.
GroupAnagrams	Group Anagrams: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Group Anagrams is to train the eye to spot the topic-level pattern quickly.
Hindex	Hindex: monotonicity is the signal that binary search applies.	The lesson is to ask whether the problem has a monotonic predicate.
HouseRobber	House Robber: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of House Robber is to train the eye to spot the topic-level pattern quickly.
JumpGame	Jump Game: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Jump Game is to train the eye to spot the topic-level pattern quickly.
KokoEatingBanana	Koko Eating Banana: monotonicity is the signal that binary search applies.	The lesson is to ask whether the problem has a monotonic predicate.
LRUCache	L R U Cache: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of L R U Cache is to train the eye to spot the topic-level pattern quickly.
LRU_cache	L R U cache: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of L R U cache is to train the eye to spot the topic-level pattern quickly.
LengthOfLongestSubstring	Length Of Longest Substring: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Length Of Longest Substring is to train the eye to spot the topic-level pattern quickly.
LongestCommonPrefix	Longest Common Prefix: contiguous range problems usually reward prefix or window state.	The lesson is to recognize when a range query can be turned into incremental state.
LongestPalindromString	Longest Palindrom String: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Longest Palindrom String is to train the eye to spot the topic-level pattern quickly.
MakeArrayZero	Make Array Zero: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Make Array Zero is to train the eye to spot the topic-level pattern quickly.
MaxProfitTrader	Max Profit Trader: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Max Profit Trader is to train the eye to spot the topic-level pattern quickly.
MaxReach	Max Reach: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Max Reach is to train the eye to spot the topic-level pattern quickly.
MaxWaterInContainer	Max Water In Container: the trick is usually an invariant that survives a single sweep.	The lesson is to reduce the problem to an invariant that can survive one pass.
MaximalSquare	Maximal Square: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Maximal Square is to train the eye to spot the topic-level pattern quickly.
MedianOf2SortedArrays	Median Of2 Sorted Arrays: the key question is which sort matches the	The lesson is to choose a sort based on data structure and constraints, not habit.

	constraints.	
MinimumPathSum	Minimum Path Sum: monotonicity is the signal that binary search applies.	The lesson is to ask whether the problem has a monotonic predicate.
MinimumSizeSubarraySum	Minimum Size Subarray Sum: monotonicity is the signal that binary search applies.	The lesson is to ask whether the problem has a monotonic predicate.
NStairsProblem	N Stairs Problem: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of N Stairs Problem is to train the eye to spot the topic-level pattern quickly.
Naukri_FlipStringToMonotone	Naukri Flip String To Monotone: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Naukri Flip String To Monotone is to train the eye to spot the topic-level pattern quickly.
NumberOfIslands	Number Of Islands: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Number Of Islands is to train the eye to spot the topic-level pattern quickly.
PlusOne	Plus One: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Plus One is to train the eye to spot the topic-level pattern quickly.
RainwaterTrapping	Rainwater Trapping: the trick is usually an invariant that survives a single sweep.	The lesson is to reduce the problem to an invariant that can survive one pass.
RansomNote	Ransom Note: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Ransom Note is to train the eye to spot the topic-level pattern quickly.
RemoveDuplicatesFromSortedArray	Remove Duplicates From Sorted Array: the key question is which sort matches the constraints.	The lesson is to choose a sort based on data structure and constraints, not habit.
RemoveDuplicatesFromSortedArray2	Remove Duplicates From Sorted Array2: the key question is which sort matches the constraints.	The lesson is to choose a sort based on data structure and constraints, not habit.
ReverseString2	Reverse String2: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Reverse String2 is to train the eye to spot the topic-level pattern quickly.
ReverseStringRecursion	Reverse String Recursion: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Reverse String Recursion is to train the eye to spot the topic-level pattern quickly.
ReverseWordsInString	Reverse Words In String: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Reverse Words In String is to train the eye to spot the topic-level pattern quickly.
RotateArray	Rotate Array: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Rotate Array is to train the eye to spot the topic-level pattern quickly.
RotatedSortedArray	Rotated Sorted Array: the key question is which sort matches the constraints.	The lesson is to choose a sort based on data structure and constraints, not habit.
SetMatrixZeros	Set Matrix Zeros: matrix/sequence problems often hinge on traversal order.	The lesson is to respect the required traversal order instead of reshaping data unnecessarily.
StackUsingLinkedList	Stack Using Linked List: identify the repeated work, choose the topic structure, and reduce the state until the	The goal of Stack Using Linked List is to train the eye to spot the topic-level pattern quickly.

	answer becomes direct.	
String_Compression	String Compression: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of String Compression is to train the eye to spot the topic-level pattern quickly.
Test	Test: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Test is to train the eye to spot the topic-level pattern quickly.
ThreeSum	Three Sum: contiguous range problems usually reward prefix or window state.	The lesson is to recognize when a range query can be turned into incremental state.
Triangle	Triangle: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Triangle is to train the eye to spot the topic-level pattern quickly.
TwoSum	Two Sum: contiguous range problems usually reward prefix or window state.	The lesson is to recognize when a range query can be turned into incremental state.
TwoSum2	Two Sum2: contiguous range problems usually reward prefix or window state.	The lesson is to recognize when a range query can be turned into incremental state.
ZigZagPattern	Zig Zag Pattern: matrix/sequence problems often hinge on traversal order.	The lesson is to respect the required traversal order instead of reshaping data unnecessarily.

linkedList

Pointer manipulation, reversals, cycle handling, merges, and list rearrangement.

Problem	Identify -> solve	Mental model
AbsoluteListSorting	Absolute List Sorting: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Absolute List Sorting is to train the eye to spot the topic-level pattern quickly.
CreateLinkedList	Create Linked List: the answer is in pointer updates, not in value sorting.	The lesson is that linked lists reward careful pointer discipline.
CycleInLinkedList	Cycle In Linked List: the answer is in pointer updates, not in value sorting.	The lesson is that linked lists reward careful pointer discipline.
DeleteMiddleOfLinkedList	Delete Middle Of Linked List: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Delete Middle Of Linked List is to train the eye to spot the topic-level pattern quickly.
DetectCycleLinkedList	Detect Cycle Linked List: the answer is in pointer updates, not in value sorting.	The lesson is that linked lists reward careful pointer discipline.
IntersectedLinkedList	Intersected Linked List: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Intersected Linked List is to train the eye to spot the topic-level pattern quickly.
Merge2SortedList_31_08	Merge2 Sorted List 31 08: the answer is in pointer updates, not in value sorting.	The lesson is that linked lists reward careful pointer discipline.
PalindromLinkedList	Palindrom Linked List: the answer is in pointer updates, not in value sorting.	The lesson is that linked lists reward careful pointer discipline.
PalindromeLinkedList_01_09	Palindrome Linked List 01 09: the answer is in pointer updates, not in value sorting.	The lesson is that linked lists reward careful pointer discipline.
RemoveDuplicate	Remove Duplicate: the answer is in pointer updates, not in value sorting.	The lesson is that linked lists reward careful pointer discipline.
ReverseInGroup	Reverse In Group: the answer is in pointer updates, not in value sorting.	The lesson is that linked lists reward careful pointer discipline.
ReverseLinkedList	Reverse Linked List: the answer is in pointer updates, not in value sorting.	The lesson is that linked lists reward careful pointer discipline.
ReverseLinkedList2	Reverse Linked List2: the answer is in pointer updates, not in value sorting.	The lesson is that linked lists reward careful pointer discipline.
RotateListByK	Rotate List By K: the answer is in pointer updates, not in value sorting.	The lesson is that linked lists reward careful pointer discipline.

main

Mixed Java practice: inheritance, interfaces, recursion, and miscellaneous interview demos.

Problem	Identify -> solve	Mental model
A	A: this folder is about Java practice patterns and demos.	The lesson is to extract the pattern behind the example, not the example itself.
AbsClass	Abs Class: this folder is about Java practice patterns and demos.	The lesson is to extract the pattern behind the example, not the example itself.
AmazonFlight	Amazon Flight: this folder is about Java practice patterns and demos.	The lesson is to extract the pattern behind the example, not the example itself.
AmazonFoodItems	Amazon Food Items: this folder is about Java practice patterns and demos.	The lesson is to extract the pattern behind the example, not the example itself.
B	B: this folder is about Java practice patterns and demos.	The lesson is to extract the pattern behind the example, not the example itself.
Child1	Child1: this folder is about Java practice patterns and demos.	The lesson is to extract the pattern behind the example, not the example itself.
Child10	Child10: this folder is about Java practice patterns and demos.	The lesson is to extract the pattern behind the example, not the example itself.
Child11	Child11: this folder is about Java practice patterns and demos.	The lesson is to extract the pattern behind the example, not the example itself.
Child12	Child12: this folder is about Java practice patterns and demos.	The lesson is to extract the pattern behind the example, not the example itself.
Child2	Child2: this folder is about Java practice patterns and demos.	The lesson is to extract the pattern behind the example, not the example itself.
Child3	Child3: this folder is about Java practice patterns and demos.	The lesson is to extract the pattern behind the example, not the example itself.
Child4	Child4: this folder is about Java practice patterns and demos.	The lesson is to extract the pattern behind the example, not the example itself.
Child5	Child5: this folder is about Java practice patterns and demos.	The lesson is to extract the pattern behind the example, not the example itself.
Child6	Child6: this folder is about Java practice patterns and demos.	The lesson is to extract the pattern behind the example, not the example itself.
Child7	Child7: this folder is about Java practice patterns and demos.	The lesson is to extract the pattern behind the example, not the example itself.
Child8	Child8: this folder is about Java practice patterns and demos.	The lesson is to extract the pattern behind the example, not the example itself.
Child9	Child9: this folder is about Java practice patterns and demos.	The lesson is to extract the pattern behind the example, not the example itself.
CreateLinkedList	Create Linked List: this folder is about Java practice patterns and demos.	The lesson is to extract the pattern behind the example, not the example

		itself.
Inter	Inter: this folder is about Java practice patterns and demos.	The lesson is to extract the pattern behind the example, not the example itself.
IntersectedLinkedList	Intersected Linked List: this folder is about Java practice patterns and demos.	The lesson is to extract the pattern behind the example, not the example itself.
MagicValue	Magic Value: this folder is about Java practice patterns and demos.	The lesson is to extract the pattern behind the example, not the example itself.
MainClass	Main Class: this folder is about Java practice patterns and demos.	The lesson is to extract the pattern behind the example, not the example itself.
MainClass1	Main Class1: this folder is about Java practice patterns and demos.	The lesson is to extract the pattern behind the example, not the example itself.
MainClass2	Main Class2: this folder is about Java practice patterns and demos.	The lesson is to extract the pattern behind the example, not the example itself.
MergeSort	Merge Sort: this folder is about Java practice patterns and demos.	The lesson is to extract the pattern behind the example, not the example itself.
Parent	Parent: this folder is about Java practice patterns and demos.	The lesson is to extract the pattern behind the example, not the example itself.
RatInAMaze	Rat In A Maze: this folder is about Java practice patterns and demos.	The lesson is to extract the pattern behind the example, not the example itself.
ReverseLinkedList	Reverse Linked List: this folder is about Java practice patterns and demos.	The lesson is to extract the pattern behind the example, not the example itself.
c	c: this folder is about Java practice patterns and demos.	The lesson is to extract the pattern behind the example, not the example itself.
test	test: this folder is about Java practice patterns and demos.	The lesson is to extract the pattern behind the example, not the example itself.

parking_lot

Low-level system design, entities, and object composition.

Problem	Identify -> solve	Mental model
ParkingLot	Parking Lot: system design starts with entity boundaries.	The lesson is to think in terms of domain objects and responsibilities.
Runner	Runner: system design starts with entity boundaries.	The lesson is to think in terms of domain objects and responsibilities.

priorityQueue

Priority-queue style selection problems, streaming top-k, and min/max heap usage.

Problem	Identify -> solve	Mental model
ConnectRopesWithMinimumCost	Connect Ropes With Minimum Cost: priority queues are for 'best candidate now' problems.	The lesson is to store only the frontier of relevance.
FindMedianOfNumberStream	Find Median Of Number Stream: priority queues are for 'best candidate now' problems.	The lesson is to store only the frontier of relevance.
KthLargestElement	Kth Largest Element: priority queues are for 'best candidate now' problems.	The lesson is to store only the frontier of relevance.
PriorityQueueSample	Priority Queue Sample: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Priority Queue Sample is to train the eye to spot the topic-level pattern quickly.
TopKFrequentElements	Top K Frequent Elements: priority queues are for 'best candidate now' problems.	The lesson is to store only the frontier of relevance.

singleton

Singleton pattern variants and lifecycle control.

Problem	Identify -> solve	Mental model
EagerSingleton	Eager Singleton: singleton means one controlled instance, not many copies.	The lesson is about lifecycle control and shared identity.
StaticEagerSingleton	Static Eager Singleton: singleton means one controlled instance, not many copies.	The lesson is about lifecycle control and shared identity.

sort

Comparison and non-comparison sorting, partitioning, and order-statistic thinking.

Problem	Identify -> solve	Mental model
CountSort	Count Sort: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Count Sort is to train the eye to spot the topic-level pattern quickly.
CountSort_String	Count Sort String: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Count Sort String is to train the eye to spot the topic-level pattern quickly.
InsertionSort	Insertion Sort: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Insertion Sort is to train the eye to spot the topic-level pattern quickly.
MergeHalfSortedArrays	Merge Half Sorted Arrays: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Merge Half Sorted Arrays is to train the eye to spot the topic-level pattern quickly.
MergeSort	Merge Sort: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Merge Sort is to train the eye to spot the topic-level pattern quickly.
MergeSort_05_08_2024	Merge Sort 05 08 2024: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Merge Sort 05 08 2024 is to train the eye to spot the topic-level pattern quickly.
QuickSort	Quick Sort: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Quick Sort is to train the eye to spot the topic-level pattern quickly.
RadixSort	Radix Sort: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Radix Sort is to train the eye to spot the topic-level pattern quickly.
SelectionSort	Selection Sort: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Selection Sort is to train the eye to spot the topic-level pattern quickly.
runner	runner: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of runner is to train the eye to spot the topic-level pattern quickly.

stack

Monotonic stack, parsing, expression-like problems, and stack-based state compression.

Problem	Identify -> solve	Mental model
AsteroidCollision	Asteroid Collision: stack state is the natural fit for nested or cancelling operations.	The lesson is that stacks are good for 'last unresolved thing first' logic.
BalancedParenthesis	Balanced Parenthesis: stack state is the natural fit for nested or cancelling operations.	The lesson is that stacks are good for 'last unresolved thing first' logic.
CelebrityProblem	Celebrity Problem: stack state is the natural fit for nested or cancelling operations.	The lesson is that stacks are good for 'last unresolved thing first' logic.
LargestHistogram	Largest Histogram: monotonic stacks turn repeated comparisons into one linear pass.	The lesson is that one-pass state compression beats repeated local comparisons.
LargestRectangle	Largest Rectangle: monotonic stacks turn repeated comparisons into one linear pass.	The lesson is that one-pass state compression beats repeated local comparisons.
MinStack	Min Stack: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Min Stack is to train the eye to spot the topic-level pattern quickly.
Min_Stack	Min Stack: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Min Stack is to train the eye to spot the topic-level pattern quickly.
NextGreaterElement	Next Greater Element: monotonic stacks turn repeated comparisons into one linear pass.	The lesson is that one-pass state compression beats repeated local comparisons.
NextGreaterElement2	Next Greater Element2: monotonic stacks turn repeated comparisons into one linear pass.	The lesson is that one-pass state compression beats repeated local comparisons.
PaintCoordinated	Paint Coordinated: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Paint Coordinated is to train the eye to spot the topic-level pattern quickly.
RemoveKDigits	Remove K Digits: stack state is the natural fit for nested or cancelling operations.	The lesson is that stacks are good for 'last unresolved thing first' logic.
SimplifyPath	Simplify Path: stack state is the natural fit for nested or cancelling operations.	The lesson is that stacks are good for 'last unresolved thing first' logic.
SlidingWindowMaximum	Sliding Window Maximum: monotonic stacks turn repeated comparisons into one linear pass.	The lesson is that one-pass state compression beats repeated local comparisons.
StackUsingArray	Stack Using Array: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.	The goal of Stack Using Array is to train the eye to spot the topic-level pattern quickly.
StockSpannerStack	Stock Spanner Stack: monotonic stacks turn repeated comparisons into one linear pass.	The lesson is that one-pass state compression beats repeated local comparisons.
StringDecode	String Decode: stack state is the natural fit for nested or cancelling operations.	The lesson is that stacks are good for 'last unresolved thing first' logic.
SumOfSubarraysMinimum	Sum Of Subarrays Minimum: identify the repeated work, choose the topic structure, and reduce the state until the	The goal of Sum Of Subarrays Minimum is to train the eye to spot the topic-level pattern quickly.

	answer becomes direct.	
--	------------------------	--

streams

Functional-style transformations and stream pipelines.

Problem	Identify -> solve	Mental model
ReverseString	Reverse String: streams are about expressing transformations cleanly.	The lesson is that streams are a data pipeline abstraction, not magic performance.
Test1	Test1: streams are about expressing transformations cleanly.	The lesson is that streams are a data pipeline abstraction, not magic performance.
User	User: streams are about expressing transformations cleanly.	The lesson is that streams are a data pipeline abstraction, not magic performance.

trie

Prefix-tree based string matching and word segmentation problems.

Problem	Identify -> solve	Mental model
InsertAndSearchInTrie	Insert And Search In Trie: tries compress repeated prefixes.	The lesson is to trade memory for repeated-prefix speed.
Trie	Trie: tries compress repeated prefixes.	The lesson is to trade memory for repeated-prefix speed.
WordBreak	Word Break: word-break is prefix DP with shared trie lookups.	The lesson is that prefix data structures pair naturally with word segmentation.
trie_	trie: tries compress repeated prefixes.	The lesson is to trade memory for repeated-prefix speed.